

ML Equivalent Control Strategy For ACC Considering Front And Rear Vehicles

CPS Application Lab Report

Submitted By

Revanth Valupadasu

25AT63R04

Advanced technology Development Centre

Indian institute of Technology,Kharagpur



15th April 2026

Under the Guidance of
Dr. Somnath Sengupta

Acknowledgment

Respected Faculty Members of Advance Technology Development Centre (ATDC), IIT Kharagpur, I am writing to express my sincere gratitude for the opportunity to work in the “CPS Application Lab” under your guidance. The experience of working in ML Equivalent Control Strategy For ACC Considering Front And Rear Vehicles has been truly valuable and enriching.

I would like to extend my heartfelt thanks to Dr. Somnath Sengupta for giving me the opportunity to work on this project and for his constant support and guidance throughout the duration of the project. His expertise and mentorship have been invaluable in shaping my understanding of the subject matter and enhancing my skills.

I would also like to express my gratitude to my Teaching Assistant Mr. Gopinath for his support and guidance during the project. His insights were instrumental in the successful completion of the project.

I am grateful to the entire faculty team of ATDC, IIT Kharagpur for organizing such a beneficial “CPS Application Lab” and for their unwavering support and encouragement throughout the Lab. The opportunity to work on this project has been a significant learning experience for me, and I am truly grateful for your guidance and mentorship.

Thank you once again for your support and for providing me with this incredible opportunity. I am privileged to have been associated with ATDC, IIT Kharagpur, and I will always cherish this experience.

Revanth Valupadasu
25AT63R04

Table of Contents

1.Abstract	3
2.Technical Background & Importance of the Problem	4
3.Objectives & Problem Definition	5
4.System Description	6
5. Working Principle	7
6. Simulation & Experimental Results along with Components used	10
7.Challenges and future work	15
8. Conclusion	16
9.References	17

1.ABSTRACT

This report presents the design, implementation, and performance analysis of a Miniature Adaptive Cruise Control (ACC) testbed built on a Raspberry Pi 4. The system investigates Machine Learning-enhanced predictive control by combining Model Predictive Control (MPC) with trained Multilayer Perceptron (MLP) networks. The key innovation lies in replacing computationally heavy online optimisation with a fast, offline-trained NumPy MLP that replicates MPC behaviour at sub-millisecond inference speeds. The system extends awareness to both front and rear traffic using dual HC-SR04 ultrasonic sensors. A cascaded dual-MPC architecture (MPC1 + MPC2) solved via Hildreth's Quadratic Programming is developed and validated in simulation; the trained MLP surrogate is subsequently deployed on hardware. Results demonstrate a 50% reduction in velocity-tracking RMSE over a TensorFlow PID baseline and R^2 values exceeding 0.98 across key model scenarios, confirming the viability of edge-deployable ML-based ACC.

2. Technical Background & Importance of the Problem

Adaptive Cruise Control (ACC) is a Level-1 ADAS feature that maintains a set speed and a safe following distance from the vehicle ahead. Classical ACC relies on Proportional-Integral-Derivative (PID) controllers, which require environment-specific manual tuning and do not account for rear-approaching traffic. Rear-end collisions constitute a significant fraction of total road accidents globally, largely attributable to insufficient headway, sudden braking, and delayed response — scenarios where a front-only ACC is structurally blind.

Model Predictive Control (MPC) offers multi-objective optimisation over a prediction horizon, gracefully handling safety constraints and vehicle dynamics. However, real-time MPC demands considerable computational resources, making it impractical for resource-constrained embedded systems. The seminal research by Sharma, Sengupta, and Ghosh (SAE Int. J. Veh. Dyn., Stab., and NVH, 9(1):121-150, 2025) formally established the cascaded dual-aware MPC framework — pairing a kinematic High-Level MPC that computes a reference speed considering both front and rear vehicles with a dynamic Low-Level MPC that generates throttle/brake commands. Their Prescan-based simulations demonstrated clear superiority of dual-aware MPC over fuzzy-logic and front-aware controllers in critical scenarios including fast rear approach and stop-and-go traffic.

The present project bridges the gap between theoretical MPC design and hardware deployment by training MLP surrogate models on MPC-generated data. The surrogates are inference-only NumPy networks, eliminating TensorFlow dependencies and enabling deployment on the Raspberry Pi GPIO environment. This approach — learning to mimic an optimal controller — is a well-established paradigm in imitation learning and approximate dynamic programming, and is here applied concretely to miniature-scale automotive control.

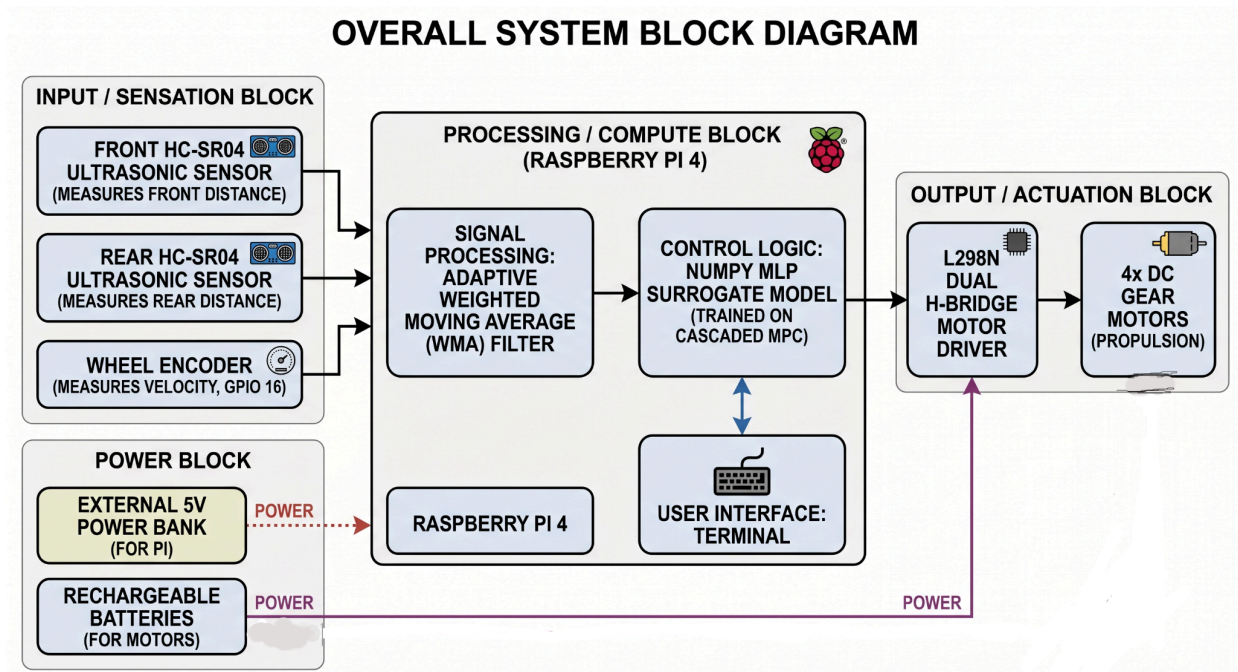
3.Objectives & Problem Definition

The project addresses four progressive objectives, each building on the previous:

- Minimum PWM Discovery: Empirically identify the threshold PWM value that overcomes the motor deadband on the car chassis, below which no motion occurs, using a Flask-based GUI calibration interface.
- ML-Based Speed Control: Replace a TensorFlow PID controller with a lightweight NumPy MLP trained on real-world telemetry data collected from the car, operating in a 4-feature input space to predict PWM output.
- Front-Aware MPC Surrogate: Simulate 8,000-point cascaded MPC trajectories (front vehicle following), train an MLP to replicate this behaviour, and deploy the surrogate on hardware.
- Dual-Aware ACC: Extend the architecture to dual ultrasonic sensing (front + rear); implement a cascaded MPC1+MPC2 solved via Hildreth's QP; train an MLP dual-surrogate and validate rear-awareness on hardware.

The overarching problem is: given the inability to run heavy optimisation solvers in real-time on a Raspberry Pi, can a pre-trained neural network faithfully approximate the control policy of a mathematically optimal MPC while remaining deployable within the hardware and latency constraints of an embedded automotive testbed?

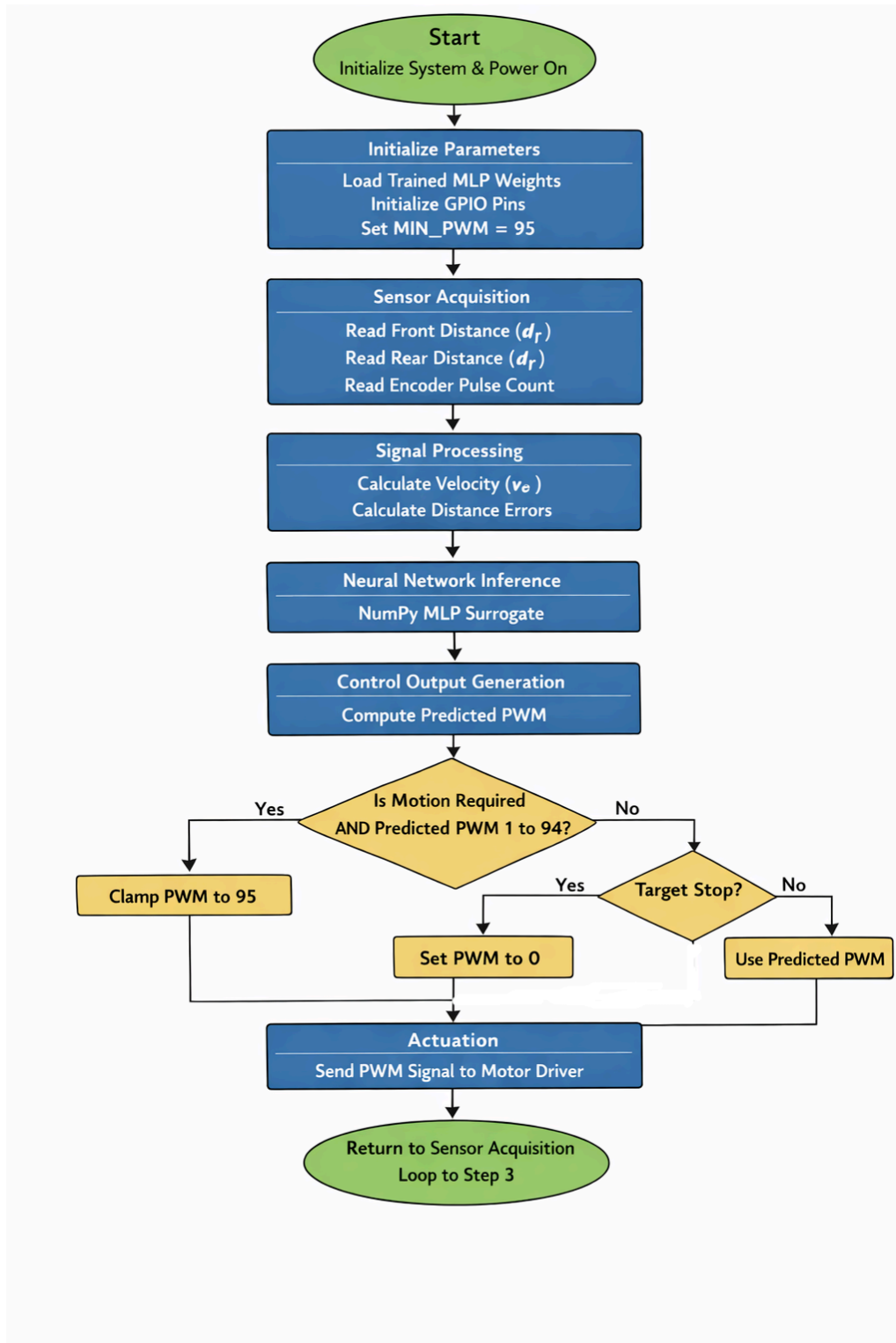
4. System Description



The Adaptive Cruise Control (ACC) testbed is designed as a sophisticated robotic vehicle employing forward and rear-facing ultrasonic sensors alongside a wheel encoder for comprehensive, real-time environmental sensing. At its core, a Raspberry Pi 4 acts as the central processing unit, utilizing an adaptive weighted moving average filter for signal smoothing and executing a custom neural network-based controller (NumPy MLP Surrogate). Finally, precise control outputs are generated for the L298N motor driver to actuate the four DC gear motors, with incorporated deadband logic ensuring reliable responsiveness.

5. Working Principle

5.1. System Algorithmic Flowchart



5.2 Safe Distance Model

Following the formulation of Sharma et al. (2025), the safe distance between the ego vehicle and a surrounding vehicle is:

$$d_{safe} = HWT_{dem} \times v_e + d_{standstill}$$

where $HWT_{dem} = 2.5$ s (demanded headway time) and $d_{standstill} = 4$ m. This dynamic safe-distance grows with speed, preventing collisions at any velocity. The distance error $\Delta d = d_{rel} - d_{safe}$ is the core feedback signal: positive means it is safe to accelerate, negative means decelerate.

5.3 Cascaded Dual-Aware MPC Architecture

- **MPC1 (High-Level):** Uses relative kinematic equations for three vehicles (front, ego, rear). Its state is the reference velocity v_{ref} ; the control input is theoretical ego acceleration a_e . The cost function penalises both Δd_f (front distance error) and Δd_r (rear distance error) with a higher weight on the front vehicle. Hildreth's Quadratic Programming solves the optimisation subject to acceleration bounds and a speed ceiling.
- **MPC2 (Low-Level):** A 3-state dynamic model (d_f, d_r, v_e) receives the v_{ref} from MPC1 as its tracking reference. It outputs a throttle-brake command in $[-1, +1]$ that balances speed tracking and smooth control transitions. The same Hildreth QP framework is used, consistent with the theoretical derivation in the source paper.

5.4 MLP Surrogate Training

For each controller scenario, simulation data (CSV) is collected. Feature engineering produces relevant inputs: current speed, target speed, front distance, rear distance (where applicable), and their error terms. A two-hidden-layer MLP is trained using mini-batch gradient descent with ReLU activations. The best model (lowest RMSE on a held-out **20%** test set) is serialised as a .npz file containing weight matrices and bias vectors.

At runtime on the Raspberry Pi, inference consists entirely of NumPy matrix multiplications and ReLU calls—no deep-learning framework is required. The TensorFlow to NumPy migration was critical: TensorFlow caused import errors in the Raspberry Pi GPIO environment. The manual reimplement of the forward pass reduced the inference footprint and enabled full on-device deployment.

5.5 Motor Deadband Handling

A systematic calibration experiment via the Flask GUI revealed that the motors produce no motion below $\text{PWM} \approx 95/255$ (approximately **37%** duty cycle). This deadband is hard-coded as MIN_{PWM} in every controller script. When the MLP or MPC outputs a value below this threshold and motion is required, the output is clamped to MIN_{PWM} . When the output is zero (intentional stop), the PWM is set to 0.

5.6 Encoder Velocity Estimation

The wheel encoder (348 counts/revolution, 65 mm wheel diameter) generates pulses on GPIO 16. Velocity is computed as:

$$v = \left(\frac{\Delta \text{counts}}{348} \right) \times \pi \times 0.065 \text{ m/s}$$

An adaptive Weighted Moving Average (WMA) filter with spike rejection (changes $> 0.3 \text{ m/s}$ in one step are discarded) and variable smoothing coefficient $\alpha \in [0.25, 0.70]$ depending on error magnitude ensures clean velocity feedback despite mechanical vibration.

6. Simulation , Experimental Results along with Components used

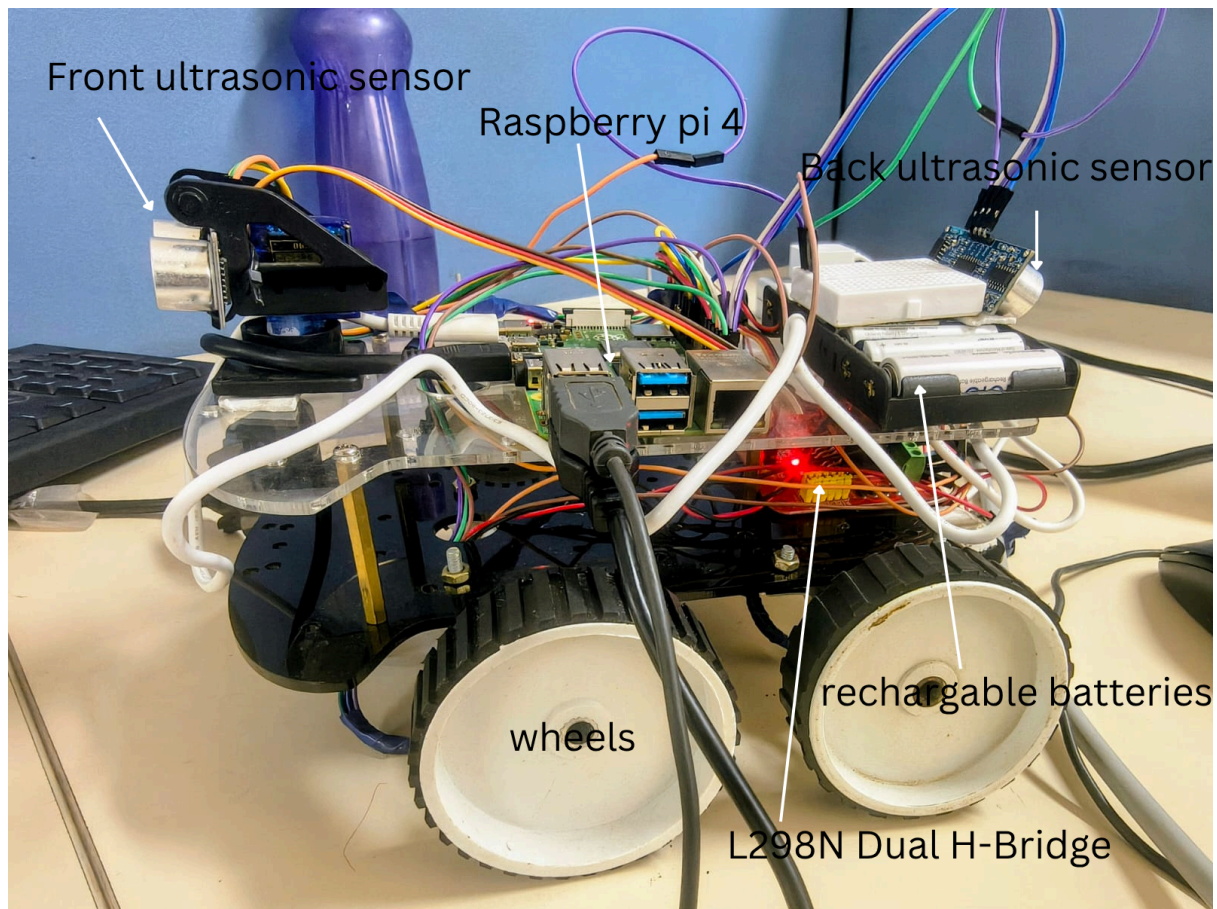


Fig1: robotic car setup

6.1 Hardware Components

The following table summarises all hardware components, their roles, and key specifications:

Component	Model / Spec	Role in System
Main Compute Unit	Raspberry Pi 4 (4 GB RAM)	Runs Python controller, Flask GUI, MLP inference
Motor Driver	L298N Dual H-Bridge	PWM motor control via GPIO 17 / 22 / 27 / 25 (ENA/ENB)
Front Ultrasonic Sensor	HC-SR04 — GPIO 23 (Trig) / 24 (Echo)	Measures distance to vehicle ahead
Rear Ultrasonic Sensor	HC-SR04 — GPIO 20 (Trig) / 21 (Echo)	Measures distance of following vehicle
Wheel Encoder	GPIO 16 — 348 counts/rev	Velocity feedback (65 mm wheel, $v = \text{counts} \times \pi \times 0.065 \text{ m}$)
Drive Motors	4× DC Gear Motors	Propulsion; min. effective PWM $\approx 95/255$ (deadband)

Component	Model / Spec	Role in System
Power Supply	External Power Bank	Portable 5 V supply for Pi
GUI Interface	Flask Web Server (Python)	Browser-based PWM slider + direction control for calibration
Power supply	Rechargeable batteries	Power supply to motor

Table 1: Hardware Components

6.2 Software & Control Pipeline

The pipeline is structured in five sequential phases that progressively increase controller sophistication:

Phase	Script	Description
Phase 1: GUI Calibration	frontvehiclewithoutus.py	Flask server with PWM slider 0–255; empirically locates $\text{MIN_PWM} \approx 95$
Phase 2: ML-PID	mlpid.py	NumPy MLP (2 hidden layers); 4 input features $\rightarrow$ PWM output; trained on real telemetry
Phase 3: Front MPC	frontmpc.py + mlfront.py	8,000-pt CSV from cascaded MPC simulation; MLP trained to mimic MPC (RMSE-optimised)
Phase 4: Dual MPC	dualaware.py	HighLevelMPC (Hildreth QP, 1-state kinematic) + LowLevelMPC (3×3 dynamics)
Phase 5: ML Dual	mldual.py	MLP dual-surrogate deployed on car; rear approach $\rightarrow$ speed increase logic

Table 2: Software & Control Pipeline

6.3 Key Performance Metrics Summary

The following table consolidates quantitative results across all model scenarios. Inference time, R^2 , RMSE, and MAE are computed on held-out test sets.

Scenario	Inference (ms)	R^2	RMSE	MAE	Train Iters	SAMPLES
Dual Sim	0.145	0.947	0.036	0.022	168	8000
Dual Speed	—	—	0.186	0.116	106	2558
Front Real	0.398	0.988	7.203	1.600	372	2540
Front Sim	0.179	0.991	7.742	2.919	494	8000
PID Real	0.138	0.946	14.637	8.186	1770	935
PID Sim	0.316	0.994	6.452	3.618	301	8000

Table 3: Key Performance Metrics Summary

Validation results demonstrate that the Neural Network successfully learned and replicated the expert mathematical controller's behavior with a near-perfect R^2 correlation

Fig 2.Trained using PID simulation data:

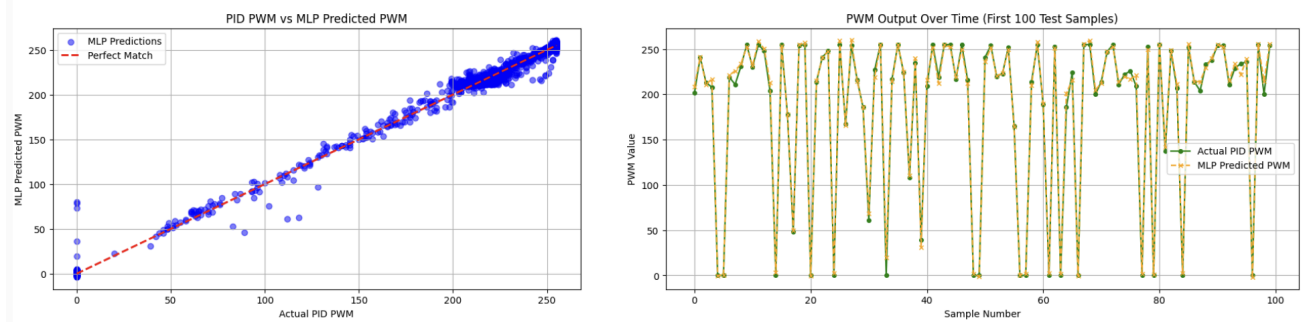


Fig 3.Trained using PID real world data:

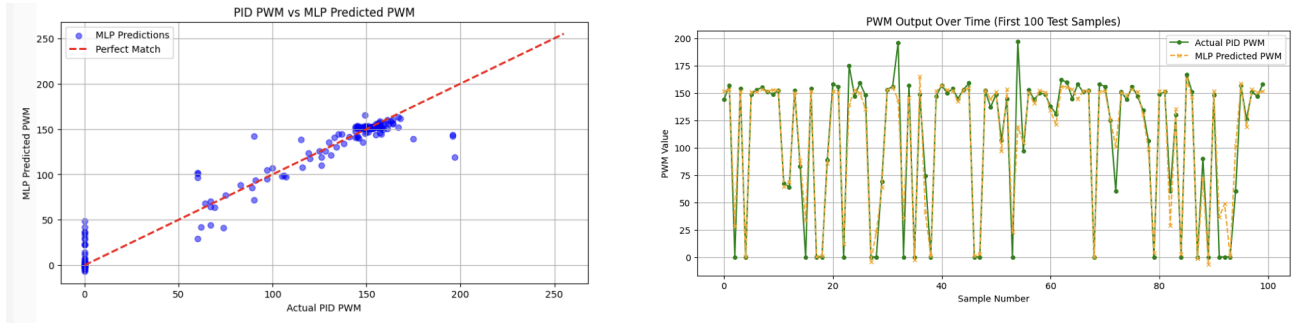


Fig 4.Trained using front aware mpc simulation data:

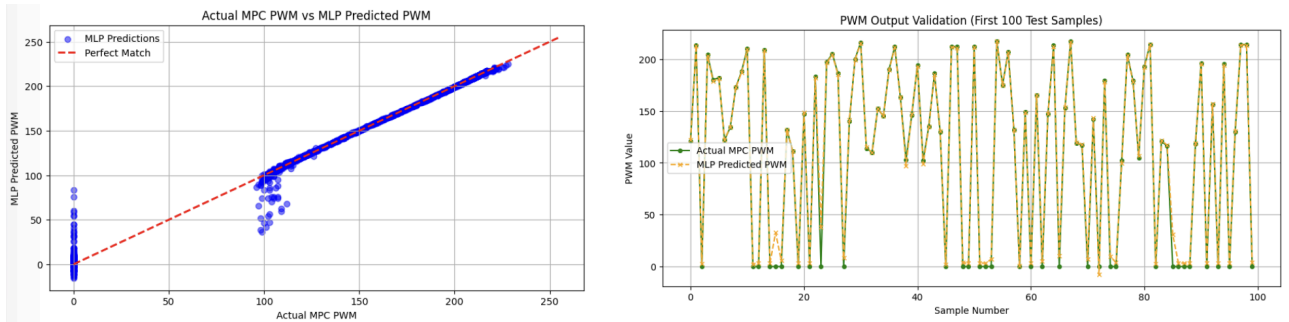


Fig 5.Trained using front aware mpc real world data:

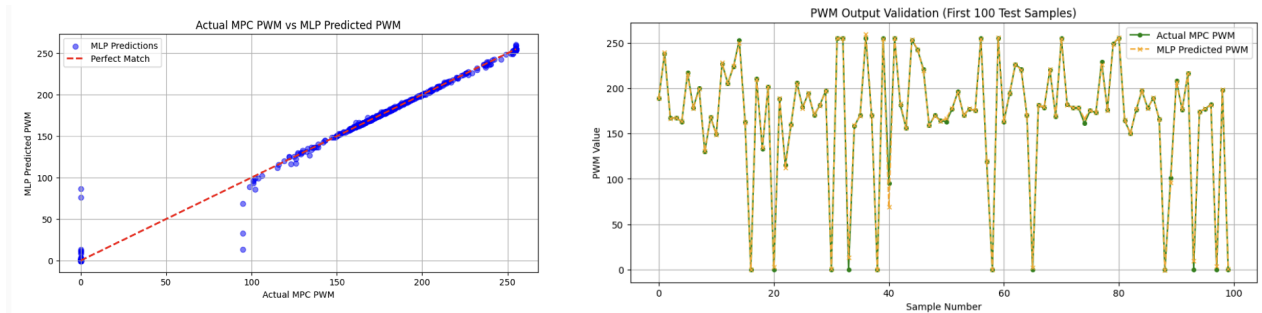


Fig 6.Trained using Dual aware mpc simulation data:

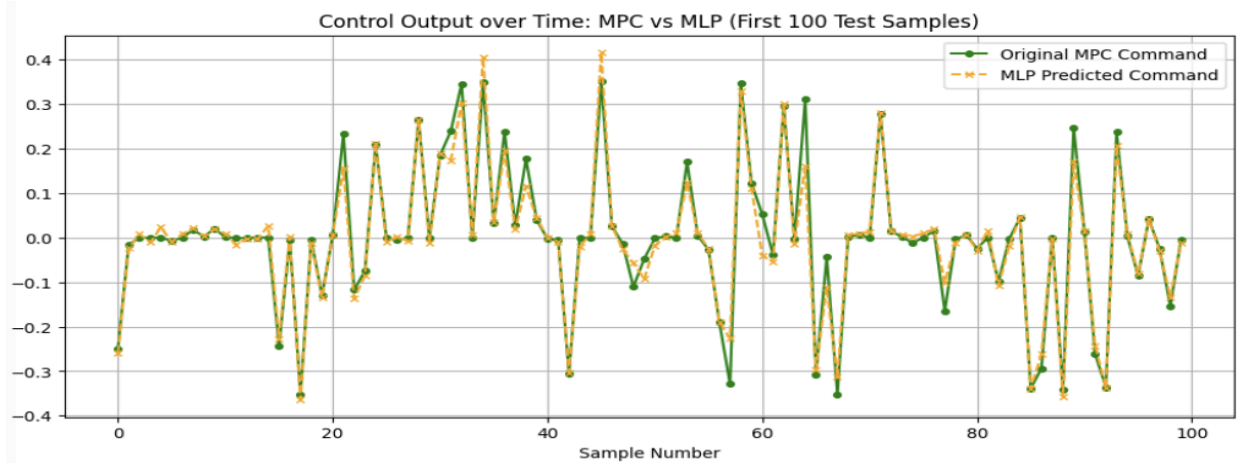
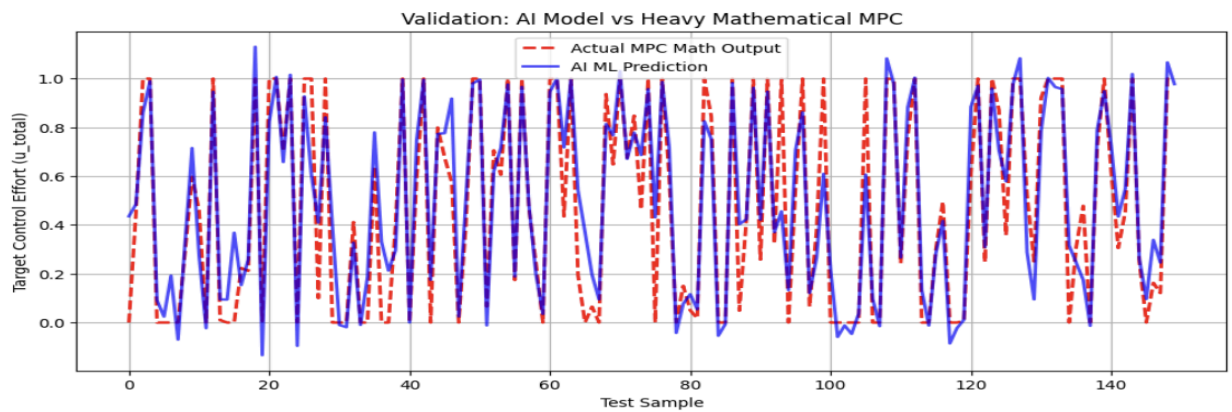


Fig 7.To test the speed increase when rear vehicle approaches:



6.4 Front-Aware MPC Surrogate

The MLP trained on 8,000 MPC simulation data points achieved $R^2 = 0.991$ (Sim) and 0.988 (Real), with inference times of 0.179 ms and 0.398 ms respectively. PWM output validation plots (first 100 test samples) show near-perfect overlap between actual MPC PWM and MLP predicted PWM, confirming that the surrogate faithfully replicates the cascaded MPC control policy. The scatter plots show tight clustering around the perfect-match diagonal for the simulation case, with slightly higher scatter in the real-world case consistent with unmodelled dynamics.

6.5 Dual-Aware MPC Surrogate

The dual-aware MLP (control output in $[-1, +1]$ normalised range) achieved $R^2 = 0.947$ with $RMSE = 0.036$ and $MAE = 0.022$, requiring only 168 training iterations. The control output over time plots demonstrate strong correspondence between the original MPC command and MLP prediction. A secondary validation comparing AI model vs. heavy mathematical MPC across 150 test samples confirms that the surrogate captures the sign, magnitude, and timing of control transitions.

7. Challenges and Future work

7.1 Strengths

- Full hardware-software co-design: the system moves from simulation all the way to on-car deployment, bridging the sim-to-real gap with systematic data collection.
- NumPy-only inference: eliminating framework dependencies makes the solution portable across any Python environment, a practical engineering decision validated in production.
- Dual-awareness principle validated: the rear-approach speed-increase logic functioned correctly on hardware, proving the concept even before dual-MPC Q/R weights are finalised.
- Encoder robustness: the adaptive WMA filter with spike rejection produced clean velocity estimates despite mechanical vibration at low PWM, a non-trivial signal-processing challenge on miniature cars.

7.2 Limitations & Open Challenges

- Dual MPC Q/R Tuning: The cascaded dualaware.py compiles and runs but optimal weight matrices for the Hildreth QP have not been experimentally validated on hardware. Simulation confirms correct architecture; hardware tuning remains as future work.
- Limited track length and absence of lane-change scenarios constrains generalisability; the system assumes straight-line motion with no overtaking.
- Ultrasonic sensors have a detection range of approximately 2–400 cm with significant noise at close range, which is more limiting than the radar/lidar modelled in Prescan.

7.3 Comparison of Control Strategies

Consistent with Sharma et al. (2025), the MPC-based strategy outperforms fuzzy-logic approaches in all critical scenarios. The key advantage of MPC is its ability to anticipate future states over the prediction horizon N_p , enabling proactive speed adjustments before a distance violation occurs. The fuzzy controller, constrained by its rule base, reacts only to the current state and produces higher jerk (3.13 vs. 2.39 m/s³ in Scenario 1) and lower minimum rear distance (36.45 vs. 38.4 m). The MLP surrogate inherits the smooth, anticipatory behaviour of MPC at a fraction of the online computational cost.

8. Conclusion

This project successfully demonstrates that ML-enhanced predictive control is a practical pathway to deploying advanced ACC functionality on resource-constrained embedded hardware. The principal contributions are:

- Construction of a fully functional miniature ACC testbed with dual ultrasonic sensing, wheel encoder feedback, and L298N motor driver on Raspberry Pi 4.
- Systematic PWM deadband characterisation ($\text{MIN_PWM} \approx 95/255$) via Flask GUI, establishing the foundation for reliable low-speed operation.
- Complete TensorFlow-to-NumPy migration of the MLP inference pipeline, achieving full on-device deployment with sub-0.4 ms inference latency.
- Design and simulation validation of a complete cascaded dual-aware MPC (MPC1 + MPC2) with Hildreth QP, consistent with the Sharma et al. (2025) framework.
- Hardware-validated dual-aware MLP controller demonstrating rear-awareness: rear vehicle approach correctly triggers ego vehicle speed increase.

Future directions include: hardware-tuning of MPC1/MPC2 Q/R weights, integration of a Reinforcement Learning policy replacing supervised MLP mimicry, RTOS-based control-loop guarantees, EKF-based sensor fusion for ultrasonic noise rejection, and extension to 3+ vehicle platoons with V2V communication simulation.

9.References

- [1] Sharma, V., Sengupta, S., and Ghosh, S., "Enhanced Adaptive Cruise Control Strategies Considering Front and Rear Vehicles," SAE Int. J. Veh. Dyn., Stab., and NVH 9(1):121-150, 2025, doi:10.4271/10-09-01-0007.
- [2] Kothuri, N., Chandrasekhar, A., and Sengupta, S., "A Novel Model Predictive Control Framework for Energy Management in Retrofit Hybrid Electric Vehicles," SAE Int. J. Elec. Veh. 12(3):343-359, 2023.